

TÌM HIỂU

XÂY DỰNG HỆ THỐNG DATA LAKEHOUSE

THEO DÕI THỊ TRƯỜNG CHỨNG KHOÁN

I. Đặt Vấn Đề & Mục Tiêu Dự Án

Thị trường chứng khoán Việt Nam hiện đang chứng kiến sự tăng trưởng mạnh về số lượng nhà đầu tư cá nhân cũng như khối lượng giao dịch. Lượng dữ liệu tài chính ngày càng phức tạp và đa dạng được sinh ra từ nhiều nguồn khác nhau. Dữ liệu thị trường chứng khoán thường tồn tại phân tán, thiếu nhất quán về cấu trúc, khó tích hợp giữa dữ liệu lịch sử (batch) và dữ liệu theo thời gian thực (streaming).

Từ đó cần thiết phải có một hệ thống lưu trữ, biến đổi thống nhất phục vụ cho mục đích phân tích tính toán các chỉ số phân tích cơ bản và chỉ số phân tích kỹ thuật như EMA, SMA, VWAP,... giúp tín hiệu giao dịch kịp thời cho các nhà đầu tư. Để giải quyết bài toán này, dự án định hướng xây dựng một hệ thống Data LakeHouse.

II. Giá Trị Thực Tế Hệ Thống Mang Lại Cho Người Dùng

Dự án hướng tới việc giải quyết triệt để các nỗi đau của nhà đầu tư thông qua 6 tính năng cụ thể:

- **Xem toàn bộ thị trường trong một chỗ:** Thay vì mở 3–4 app, người dùng vào một dashboard thấy ngay: mã nào tăng mạnh nhất hôm nay, ngành nào đang dẫn dắt, VN-Index đang ở vùng nào so với lịch sử.
- **Tín hiệu kỹ thuật tự động – không cần tự tính:** Hệ thống tính sẵn RSI, MACD, Bollinger Band cho tất cả mã. Người dùng chỉ cần nhìn (ví dụ: RSI của VCB đang 72 — có thể quá mua) để cân nhắc chốt lời.
- **Cảnh báo real-time khi cần hành động:** Thay vì nhìn màn hình cả ngày, người dùng đặt ngưỡng trước (ví dụ: "Báo khi VCB về 84,000" hoặc "Báo khi RSI vượt 70") và hệ thống tự nhắn tin/email kịp thời.
- **Tra cứu lịch sử nhanh:** ClickHouse trả kết quả truy vấn lịch sử (ví dụ: VCB 3 tháng qua biến động thế nào) trong vài mili-giây thay vì chờ web load.
- **Hỏi đáp Text-to-SQL (demo):** Hỏi đáp dạng Text-to-SQL (proof-of-concept): người dùng gõ câu hỏi tự nhiên (ví dụ: "mã nào RSI thấp nhất hôm nay?"), hệ thống chuyển thành câu SQL, truy vấn

ClickHouse và trả về kết quả. Pipeline: FastAPI nhận câu hỏi → LLM (GPT-4o mini) sinh SQL → thực thi trên ClickHouse → trả lời. Phạm vi demo, không tích hợp LLM phức tạp.

- **Dữ liệu đáng tin cậy, nhất quán:** Nhờ Great Expectations kiểm tra chất lượng, người dùng không gặp tình trạng dữ liệu bị thiếu ngày, sai giá, hoặc khối lượng âm, tránh quyết định sai lầm.

III. Chi Tiết Luồng Dữ Liệu (Input & Output)

3.1. Đầu vào (Input) của hệ thống

Nguồn batch (theo lịch, mỗi ngày sau 15:00 khi sàn đóng cửa):

Nguồn	Dữ liệu thu thập được	Ví dụ cụ thể
vnstock	OHLCV — giá mở, cao, thấp, đóng, khối lượng theo ngày	VCB: Open=85k, High=87k, Low=84.5k, Close=86k, Vol=2.1M
Finnhub	Tin tức, chỉ số tài chính cơ bản (P/E, EPS, ROE)	VCB P/E = 12.4, tin tức: "VCB báo lãi Q2..."
API niêm yết	Danh sách mã cổ phiếu, tên công ty, ngành, vốn hóa	VCB — Vietcombank — Ngân hàng — Vốn hóa 300k tỷ
VN-Index	Giá trị chỉ số tổng hợp theo ngày	VN-Index: 1,280.5 điểm

Nguồn streaming (live 9:00 – 14:45 trong giờ giao dịch):

Nguồn	Dữ liệu thu thập được	Ví dụ cụ thể
DNSE WebSocket API	Sổ lệnh (order book), từng lệnh mua/bán theo giây	10:15:32 — VCB mua 1,000 cp @ 86,500đ

3.2. Kiến Trúc Phân Tầng Medallion (Bronze / Silver / Gold)

Hệ thống áp dụng đúng chuẩn Medallion Architecture với 3 tầng độc lập, mỗi tầng có trách nhiệm rõ ràng:

Tầng	Mục đích	Công cụ	Ví dụ dữ liệu
Bronze	Lưu dữ liệu thô 100% nguyên bản từ nguồn, không chỉnh sửa	MinIO (Parquet)	raw/2024-01-15/vcb_ohlc.parquet
Silver	Làm sạch, cast kiểu, chuẩn hóa tên cột — KHÔNG tính chỉ số nghiệp vụ	Polars + Great Expectations	Loại giá âm, null, high < low
Gold	Tính toán chỉ số kỹ thuật (SMA/EMA/RSI/MACD), tổng hợp phục vụ use case cụ thể	dbt (SQL trong ClickHouse)	fact_daily_price với EMA_12, RSI_14...

Lưu ý quan trọng: Việc tính chỉ số kỹ thuật (SMA, EMA, MACD, RSI, Bollinger Bands) được thực hiện hoàn toàn ở tầng Gold bằng dbt models — KHÔNG tính ở Silver. Silver chỉ làm sạch dữ liệu. Điều này đảm bảo đúng chuẩn Medallion Architecture và giúp lineage graph của dbt phản ánh chính xác luồng biến đổi dữ liệu.

3.3. Thiết kế Star Schema trong ClickHouse

Dữ liệu ở tầng Gold được tổ chức theo mô hình **Star Schema logic** nhằm phục vụ các truy vấn phân tích OLAP như theo dõi biến động giá, phân tích chỉ báo kỹ thuật, so sánh hiệu suất theo ngành, giám sát realtime, lưu lịch sử cảnh báo và hiển thị dashboard cho người dùng.

Do hệ thống sử dụng **ClickHouse** làm kho dữ liệu phân tích tốc độ cao, mô hình Star Schema được thiết kế theo hướng linh hoạt. Hệ thống vẫn giữ các bảng Dimension để mô tả thực thể nghiệp vụ, nhưng có thể **denormalize một số trường thường xuyên truy vấn** vào bảng fact hoặc materialized view để giảm số lượng JOIN, từ đó tăng tốc độ truy vấn dashboard.

3.3.1. Tổng quan mô hình Star Schema

Mô hình dữ liệu Gold bao gồm hai nhóm bảng chính:

- **Dimension Tables:** lưu thông tin mô tả, dùng để lọc, phân nhóm và phân tích theo ngữ cảnh.
- **Fact Tables:** lưu dữ liệu sự kiện và các chỉ số đo lường, phục vụ trực tiếp cho dashboard, cảnh báo và truy vấn phân tích.

Các bảng Dimension

Bảng	Khóa chính	Mục đích
<code>dim_date</code>	<code>date_id</code>	Lưu thông tin ngày, tháng, quý, năm và trạng thái ngày giao dịch
<code>dim_sector</code>	<code>sector_id</code>	Lưu thông tin ngành, nhóm ngành phục vụ phân tích theo sector
<code>dim_ticker</code>	<code>ticker</code>	Lưu thông tin cơ bản của mã cổ phiếu niêm yết
<code>dim_index</code>	<code>index_id</code>	Lưu thông tin các chỉ số thị trường như VN-Index, HNX-Index, VN30

Các bảng Fact

Bảng	Granularity	Mục đích
<code>fact_daily_price</code>	1 mã cổ phiếu $\times$ 1 ngày	Lưu dữ liệu OHLCV, chỉ báo kỹ thuật và vốn hóa theo ngày
<code>fact_realtime_vwap</code>	1 mã cổ phiếu $\times$ 1 phút	Lưu dữ liệu realtime, VWAP theo phút và VWAP lũy kế phiên

<code>fact_market_index</code>	1 chỉ số thị trường × 1 ngày	Lưu dữ liệu chỉ số thị trường và độ rộng thị trường
<code>fact_news_sentiment_daily</code>	1 mã cổ phiếu × 1 ngày	Lưu dữ liệu tổng hợp tin tức, cảm xúc và số nguồn tin
<code>fact_alert_event</code>	1 cảnh báo được kích hoạt	Lưu lịch sử các cảnh báo đã phát cho người dùng

3.3.2. Bảng Dimension

a. `dim_date` — Dimension thời gian

Bảng `dim_date` lưu thông tin thời gian chuẩn, phục vụ phân tích dữ liệu theo ngày, tháng, quý, năm và xác định ngày giao dịch.

Cột	Kiểu dữ liệu	Mô tả
<code>date_id</code>	UInt32	Khóa chính, định dạng YYYYMMDD
<code>date</code>	Date	Ngày thực tế
<code>year</code>	UInt16	Năm
<code>quarter</code>	UInt8	Quý

month	UInt8	Tháng
week	UInt8	Tuần trong năm
day_of_week	LowCardinality(String)	Thứ trong tuần
is_trading_day	UInt8	Đánh dấu ngày có giao dịch hay không

Bảng này giúp dashboard dễ dàng lọc dữ liệu theo khoảng thời gian, so sánh cùng kỳ, tính toán biến động theo tháng/quý/năm.

b. dim_sector — Dimension ngành

Bảng **dim_sector** lưu thông tin ngành kinh tế hoặc nhóm ngành của doanh nghiệp niêm yết.

Cột	Kiểu dữ liệu	Mô tả
sector_id	String	Khóa chính của ngành
sector_name	LowCardinality(String)	Tên ngành, ví dụ: Ngân hàng, Bất động sản, Chứng khoán
industry_group	LowCardinality(String)	Nhóm ngành cấp cao hơn

<code>description</code>	String	Mô tả ngành
--------------------------	--------	-------------

Bảng này phục vụ các phân tích như so sánh hiệu suất giữa các ngành, tạo heatmap theo ngành, lọc danh sách cổ phiếu theo nhóm ngành.

c. `dim_stock` — Dimension cổ phiếu

Bảng `dim_stock` lưu thông tin định danh và thông tin mô tả cơ bản của từng mã cổ phiếu.

Cột	Kiểu dữ liệu	Mô tả
<code>ticker</code>	String	Mã cổ phiếu, khóa chính
<code>company_name</code>	String	Tên công ty niêm yết
<code>exchange</code>	LowCardinality(String)	Sàn giao dịch: HOSE, HNX, UPCOM
<code>sector_id</code>	String	Khóa ngoại liên kết với <code>dim_sector</code>
<code>listed_date</code>	Nullable(Date)	Ngày niêm yết
<code>status</code>	LowCardinality(String)	Trạng thái: đang giao dịch, hủy niêm yết, tạm ngừng giao dịch

<code>shares_outstanding</code>	UInt64	Số lượng cổ phiếu đang lưu hành
<code>free_float_rate</code>	Nullable(Float64)	Tỷ lệ cổ phiếu tự do chuyển nhượng, chỉ điền khi nguồn dữ liệu cung cấp
<code>market_cap_latest</code>	Nullable(Float64)	Vốn hóa gần nhất, phục vụ lọc nhanh
<code>pe_latest</code>	Nullable(Float64)	P/E gần nhất
<code>eps_latest</code>	Nullable(Float64)	EPS gần nhất
<code>roe_latest</code>	Nullable(Float64)	ROE gần nhất
<code>roa_latest</code>	Nullable(Float64)	ROA gần nhất
<code>updated_at</code>	DateTime	Thời điểm cập nhật thông tin

Lưu ý: Các chỉ số như `market_cap_latest`, `pe_latest`, `eps_latest`, `roe_latest`, `roa_latest` được lưu dưới dạng **latest snapshot** để phục vụ dashboard nhanh. Tuy nhiên, để tính vốn hóa theo từng ngày, hệ thống sử dụng công thức:

$$\text{market_cap} = \text{close} * \text{shares_outstanding}$$

Trong đó `close` lấy từ `fact_daily_price`, còn `shares_outstanding` lấy từ `dim_stock` hoặc từ snapshot gần nhất tại thời điểm tính toán.

Trường `free_float_rate` là trường mở rộng và có thể để `NULL` vì không phải nguồn dữ liệu nào cũng cung cấp đầy đủ tỷ lệ cổ phiếu tự do chuyển nhượng cho thị trường Việt Nam. Trường này chỉ được cập nhật khi nguồn dữ liệu hỗ trợ.

Nếu hệ thống mở rộng theo hướng phân tích báo cáo tài chính theo từng quý/năm, các chỉ số tài chính có thể được tách sang bảng fact riêng như `fact_financial_metric`.

d. `dim_index` — Dimension chỉ số thị trường

Bảng `dim_index` lưu danh mục các chỉ số thị trường được theo dõi.

Cột	Kiểu dữ liệu	Mô tả
<code>index_id</code>	String	Khóa chính của chỉ số
<code>index_name</code>	LowCardinality(String)	Tên chỉ số: VN-Index, HNX-Index, VN30
<code>exchange</code>	LowCardinality(String)	Sàn hoặc nhóm thị trường tương ứng
<code>description</code>	String	Mô tả chỉ số

Bảng này giúp chuẩn hóa dữ liệu chỉ số thị trường và tránh lưu trực tiếp tên chỉ số lặp lại trong bảng fact.

3.3.3. Bảng Fact

a. `fact_daily_price` — Giá cổ phiếu và chỉ báo kỹ thuật cuối ngày

Đây là bảng fact trung tâm của hệ thống, lưu dữ liệu giá giao dịch theo ngày, các chỉ báo kỹ thuật và các chỉ số phục vụ dashboard.

Cột	Kiểu dữ liệu	Mô tả
<code>ticker</code>	String	Mã cổ phiếu, khóa ngoại liên kết <code>dim_stock</code>
<code>date_id</code>	UInt32	Khóa ngoại liên kết <code>dim_date</code>
<code>trading_date</code>	Date	Ngày giao dịch
<code>open</code>	Float64	Giá mở cửa
<code>high</code>	Float64	Giá cao nhất
<code>low</code>	Float64	Giá thấp nhất
<code>close</code>	Float64	Giá đóng cửa
<code>volume</code>	UInt64	Khối lượng giao dịch
<code>value</code>	Float64	Giá trị giao dịch

shares_outstanding	UInt64	Số lượng cổ phiếu đang lưu hành tại thời điểm tính toán
market_cap	Float64	Vốn hóa theo ngày, tính bằng <code>close</code> × <code>shares_outstanding</code>
price_change	Float64	Mức thay đổi giá tuyệt đối
pct_change	Float64	Mức thay đổi giá theo phần trăm
sma_20	Nullable(Float64)	Đường trung bình động 20 ngày
ema_12	Nullable(Float64)	EMA 12 ngày
ema_26	Nullable(Float64)	EMA 26 ngày
macd	Nullable(Float64)	Chỉ báo MACD
macd_signal	Nullable(Float64)	Đường tín hiệu MACD
rsi_14	Nullable(Float64)	Chỉ báo RSI 14 ngày

bb_upper	Nullable(Float64)	Dải Bollinger trên
bb_middle	Nullable(Float64)	Dải Bollinger giữa
bb_lower	Nullable(Float64)	Dải Bollinger dưới
volume_sma_20	Nullable(Float64)	Trung bình khối lượng 20 ngày
overbought_flag	UInt8	Đánh dấu RSI > 70
oversold_flag	UInt8	Đánh dấu RSI < 30
breakout_flag	UInt8	Đánh dấu giá đóng cửa vượt Bollinger Band trên
breakdown_flag	UInt8	Đánh dấu giá đóng cửa thủng Bollinger Band dưới
created_at	DateTime	Thời điểm ghi dữ liệu
updated_at	DateTime	Thời điểm cập nhật dữ liệu

Granularity: Một dòng tương ứng với một mã cổ phiếu trong một ngày giao dịch.

Mục đích sử dụng:

Bảng này phục vụ các dashboard chính như biểu đồ nến, phân tích kỹ thuật, bảng xếp hạng cổ phiếu tăng/giảm, top thanh khoản, top vốn hóa, bộ lọc cổ phiếu quá mua/quá bán và phát hiện tín hiệu kỹ thuật.

Gợi ý thiết kế vật lý trong ClickHouse:

ENGINE = MergeTree

PARTITION BY toYYYYMM(trading_date)

ORDER BY (ticker, trading_date)

Lý do lựa chọn:

- **PARTITION BY toYYYYMM(trading_date)** phù hợp với dữ liệu lịch sử theo ngày, giúp quản lý dữ liệu theo tháng.
- **ORDER BY (ticker, trading_date)** tối ưu cho truy vấn phổ biến: xem lịch sử giá và chỉ báo kỹ thuật của một mã cổ phiếu theo thời gian.
- Thiết kế này phù hợp với dashboard Stock Detail, Technical Signal Scanner và các truy vấn ad-hoc theo từng mã.

b. `fact_realtime_vwap` — Dữ liệu realtime VWAP theo phút

Bảng `fact_realtime_vwap` lưu dữ liệu tổng hợp từ luồng streaming theo từng phút. Dữ liệu được tạo từ Kafka thông qua ClickHouse Kafka Engine và Materialized View.

Cột	Kiểu dữ liệu	Mô tả
<code>ticker</code>	String	Mã cổ phiếu

minute_ts	DateTime	Timestamp đầu phút
trading_date	Date	Ngày giao dịch
open_price	Float64	Giá đầu tiên trong phút
high_price	Float64	Giá cao nhất trong phút
low_price	Float64	Giá thấp nhất trong phút
close_price	Float64	Giá cuối cùng trong phút
vwap_1m	Float64	VWAP của riêng phút hiện tại
session_vwap	Float64	VWAP lũy kế từ đầu phiên đến phút hiện tại
total_volume	UInt64	Tổng khối lượng trong phút
total_value	Float64	Tổng giá trị giao dịch trong phút
session_volume	UInt64	Tổng khối lượng lũy kế từ đầu phiên đến phút hiện tại
session_value	Float64	Tổng giá trị giao dịch lũy kế từ đầu phiên đến phút hiện tại

<code>avg_price</code>	Float64	Giá trung bình trong phút
<code>trade_count</code>	UInt32	Số lượng lệnh khớp trong phút
<code>price_vs_vwap_pct</code>	Float64	Tỷ lệ lệch giữa giá cuối phút và VWAP 1 phút
<code>price_vs_session_vwap_pct</code>	Float64	Tỷ lệ lệch giữa giá cuối phút và VWAP lũy kế phiên
<code>created_at</code>	DateTime	Thời điểm ghi dữ liệu

Granularity: Một dòng tương ứng với một mã cổ phiếu trong một phút giao dịch.

Mục đích sử dụng:

Bảng này phục vụ dashboard realtime, theo dõi giá so với VWAP, phát hiện các mã có biến động bất thường trong phiên và cung cấp dữ liệu cho Alert Engine.

Công thức tính VWAP theo phút:

$$\text{vwap_1m} = \text{total_value} / \text{total_volume}$$

Công thức tính VWAP lũy kế phiên:

$$\text{session_vwap} = \text{session_value} / \text{session_volume}$$

Công thức tính tỷ lệ lệch so với VWAP lũy kế phiên:

$$\text{price_vs_session_vwap_pct} = (\text{close_price} - \text{session_vwap}) / \text{session_vwap} * 100$$

Trong đó, **session_vwap** quan trọng hơn **vwap_1m** đối với Alert Engine vì nó phản ánh giá trung bình có trọng số theo khối lượng từ đầu phiên giao dịch đến thời điểm hiện tại.

Gợi ý thiết kế vật lý trong ClickHouse:

ENGINE = MergeTree

PARTITION BY toYYYYMMDD(trading_date)

ORDER BY (ticker, minute_ts)

Lý do lựa chọn:

- **fact_realtime_vwap** có granularity theo phút nên số lượng dòng phát sinh mỗi ngày tương đối lớn. Với khoảng 1.700 mã và khoảng 300 phút giao dịch mỗi ngày, bảng có thể phát sinh xấp xỉ 500.000 dòng/ngày.
- **PARTITION BY toYYYYMMDD(trading_date)** giúp tách dữ liệu realtime theo từng ngày giao dịch, thuận tiện cho truy vấn intraday và quản lý dữ liệu theo phiên.
- **ORDER BY (ticker, minute_ts)** phù hợp với truy vấn phổ biến: xem diễn biến VWAP của một mã cổ phiếu theo thời gian trong ngày.

c. **fact_market_index** — Chỉ số thị trường và độ rộng thị trường

Bảng **fact_market_index** lưu diễn biến các chỉ số thị trường như VN-Index, HNX-Index, VN30 và các chỉ số đo lường độ rộng thị trường.

Cột	Kiểu dữ liệu	Mô tả
index_id	String	Khóa ngoại liên kết dim_index
date_id	UInt32	Khóa ngoại liên kết dim_date

trading_date	Date	Ngày giao dịch
open_point	Float64	Điểm mở cửa
high_point	Float64	Điểm cao nhất
low_point	Float64	Điểm thấp nhất
close_point	Float64	Điểm đóng cửa
point_change	Float64	Mức thay đổi điểm
pct_change	Float64	Mức thay đổi theo phần trăm
total_volume	UInt64	Tổng khối lượng giao dịch
total_value	Float64	Tổng giá trị giao dịch
advance_count	UInt32	Số mã tăng giá
decline_count	UInt32	Số mã giảm giá
unchanged_count	UInt32	Số mã đứng giá

<code>advance_decline_ratio</code>	Float64	Tỷ lệ số mã tăng/số mã giảm
------------------------------------	---------	-----------------------------

<code>sma_20</code>	Nullable(Float64)	Trung bình động 20 ngày của chỉ số
---------------------	-------------------	------------------------------------

<code>rsi_14</code>	Nullable(Float64)	RSI 14 ngày của chỉ số
---------------------	-------------------	------------------------

<code>created_at</code>	DateTime	Thời điểm ghi dữ liệu
-------------------------	----------	-----------------------

Granularity: Một dòng tương ứng với một chỉ số thị trường trong một ngày giao dịch.

Mục đích sử dụng:

Bảng này phục vụ dashboard tổng quan thị trường, hiển thị diễn biến VN-Index, VN30, tổng thanh khoản, số lượng mã tăng/giảm và đánh giá sức khỏe chung của thị trường.

Gợi ý thiết kế vật lý trong ClickHouse:

ENGINE = MergeTree

PARTITION BY toYYYYMM(trading_date)

ORDER BY (index_id, trading_date)

Lý do lựa chọn:

- `PARTITION BY toYYYYMM(trading_date)` phù hợp với dữ liệu chỉ số theo ngày.
- `ORDER BY (index_id, trading_date)` tối ưu truy vấn lịch sử của một chỉ số thị trường như VN-Index hoặc VN30 theo thời gian.

d. `fact_news_sentiment_daily` — Tổng hợp tin tức và cảm xúc theo ngày

Bảng `fact_news_sentiment_daily` lưu kết quả tổng hợp tin tức đã xử lý NLP theo từng mã cổ phiếu và từng ngày.

Cột	Kiểu dữ liệu	Mô tả
<code>ticker</code>	String	Mã cổ phiếu
<code>date_id</code>	UInt32	Khóa ngoại liên kết <code>dim_date</code>
<code>news_date</code>	Date	Ngày ghi nhận tin tức
<code>news_count</code>	UInt32	Tổng số tin liên quan đến mã cổ phiếu
<code>source_count</code>	UInt8	Số nguồn báo khác nhau đề cập đến mã cổ phiếu
<code>positive_count</code>	UInt32	Số tin tích cực
<code>negative_count</code>	UInt32	Số tin tiêu cực
<code>neutral_count</code>	UInt32	Số tin trung lập
<code>avg_sentiment_score</code>	Float64	Điểm cảm xúc trung bình

top_headline String Tiêu đề nổi bật trong ngày

created_at DateTime Thời điểm ghi dữ liệu

Granularity: Một dòng tương ứng với một mã cổ phiếu trong một ngày.

Mục đích sử dụng:

Bảng này phục vụ dashboard tin tức và sentiment, giúp người dùng theo dõi mức độ quan tâm của truyền thông đối với từng mã cổ phiếu và đánh giá tác động cảm xúc của tin tức lên biến động giá.

Việc bổ sung **source_count** giúp hệ thống phân biệt giữa trường hợp nhiều tin đến từ cùng một nguồn và trường hợp nhiều nguồn độc lập cùng đề cập đến một mã cổ phiếu. Ví dụ, 5 tin từ 1 nguồn có mức độ lan tỏa khác với 5 tin từ 5 nguồn khác nhau.

Trong phạm vi demo, hệ thống lưu dữ liệu sentiment ở mức tổng hợp theo ngày. Nếu mở rộng, có thể bổ sung bảng chi tiết **fact_news_article** để lưu từng bài viết riêng lẻ.

Gợi ý thiết kế vật lý trong ClickHouse:

ENGINE = MergeTree

PARTITION BY toYYYYMM(news_date)

ORDER BY (ticker, news_date)

Lý do lựa chọn:

- **PARTITION BY toYYYYMM(news_date)** phù hợp với dữ liệu tin tức tổng hợp theo ngày.
- **ORDER BY (ticker, news_date)** tối ưu truy vấn lịch sử sentiment của một mã cổ phiếu theo thời gian.

e. **fact_alert_event** — Lịch sử cảnh báo đã kích hoạt

Bảng `fact_alert_event` lưu lịch sử các cảnh báo đã được kích hoạt bởi Alert Engine. Khác với bảng `user_alerts` trong PostgreSQL dùng để lưu cấu hình ngưỡng cảnh báo của người dùng, bảng `fact_alert_event` được lưu trong ClickHouse để phục vụ truy vấn nhanh theo thời gian và hiển thị dashboard lịch sử cảnh báo.

Cột	Kiểu dữ liệu	Mô tả
<code>alert_id</code>	UUID	Khóa chính của cảnh báo
<code>user_id</code>	String	Người dùng nhận cảnh báo
<code>ticker</code>	String	Mã cổ phiếu, khóa ngoại liên kết <code>dim_stock</code>
<code>date_id</code>	UInt32	Khóa ngoại liên kết <code>dim_date</code>
<code>triggered_at</code>	DateTime	Thời điểm cảnh báo được kích hoạt
<code>condition_type</code>	LowCardinality(String)	Loại điều kiện: RSI_ABOVE, RSI_BELOW, BB_BREAK, PRICE_BELOW, PRICE_ABOVE, VWAP_DEVIATION
<code>threshold_value</code>	Float64	Ngưỡng cảnh báo người dùng đặt
<code>actual_value</code>	Float64	Giá trị thực tế tại thời điểm kích hoạt

<code>channel</code>	LowCardinality(String)	Kênh gửi cảnh báo: TELEGRAM, EMAIL
<code>is_sent</code>	UInt8	Đánh dấu cảnh báo đã gửi thành công hay chưa
<code>sent_at</code>	Nullable(DateTime)	Thời điểm gửi cảnh báo
<code>created_at</code>	DateTime	Thời điểm ghi log cảnh báo

Granularity: Một dòng tương ứng với một lần cảnh báo được kích hoạt.

Mục đích sử dụng:

Bảng này phục vụ các use case sau:

- Hiển thị lịch sử cảnh báo của người dùng trên dashboard.
- Thống kê số lượng cảnh báo theo mã cổ phiếu, theo ngày, theo loại điều kiện.
- Kiểm tra trạng thái gửi cảnh báo qua Telegram hoặc Email.
- Hỗ trợ cơ chế chống spam cảnh báo bằng cách kiểm tra lần kích hoạt gần nhất.

Các trường `condition_type` và `channel` được khai báo là `LowCardinality(String)` vì đây là các trường có số lượng giá trị nhỏ và lặp lại nhiều lần. Cách khai báo này giúp ClickHouse tối ưu nén dữ liệu và tăng tốc các truy vấn lọc/nhóm theo loại cảnh báo hoặc kênh gửi.

Gợi ý thiết kế vật lý trong ClickHouse:

ENGINE = MergeTree

PARTITION BY toYYYYMM(triggered_at)

ORDER BY (user_id, triggered_at)

Lý do lựa chọn:

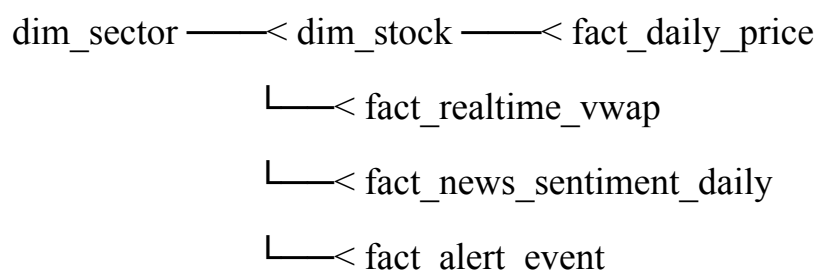
- **PARTITION BY toYYYYMM(triggered_at)** giúp quản lý dữ liệu cảnh báo theo tháng, phù hợp với dữ liệu log/sự kiện tăng dần theo thời gian.
 - **ORDER BY (user_id, triggered_at)** tối ưu cho truy vấn phổ biến nhất: xem lịch sử cảnh báo của một người dùng cụ thể theo thời gian.
 - Bảng **fact_alert_event** được lưu trong ClickHouse vì đây là dữ liệu sự kiện phục vụ phân tích và dashboard lịch sử. Trong khi đó, bảng **user_alerts** trong PostgreSQL chỉ lưu cấu hình ngưỡng cảnh báo của người dùng và được cập nhật thường xuyên.
-

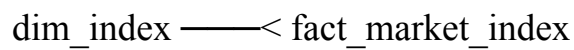
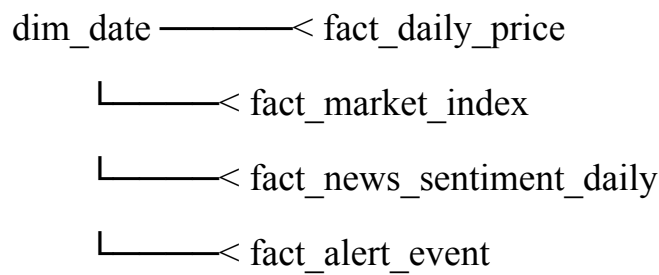
3.3.4. Quan hệ giữa các bảng

Các quan hệ chính trong Star Schema:

- **dim_sector** liên kết với **dim_stock** thông qua **sector_id**.
- **dim_stock** liên kết với **fact_daily_price** thông qua **ticker**.
- **dim_stock** liên kết với **fact_realtime_vwap** thông qua **ticker**.
- **dim_stock** liên kết với **fact_news_sentiment_daily** thông qua **ticker**.
- **dim_stock** liên kết với **fact_alert_event** thông qua **ticker**.
- **dim_date** liên kết với **fact_daily_price** thông qua **date_id**.
- **dim_date** liên kết với **fact_market_index** thông qua **date_id**.
- **dim_date** liên kết với **fact_news_sentiment_daily** thông qua **date_id**.
- **dim_date** liên kết với **fact_alert_event** thông qua **date_id**.
- **dim_index** liên kết với **fact_market_index** thông qua **index_id**.

Mô hình quan hệ tổng quát:





3.3.5. Tổng hợp thiết kế vật lý cho các bảng fact

Bảng	Partition	Order By	Lý do
fact_daily_price	toYYYYMM(trading_date)	(ticker, trading_date)	Tối ưu truy vấn lịch sử giá theo từng mã
fact_realtime_vwap	toYYYYMMDD(trading_date)	(ticker, minute_ts)	Tối ưu truy vấn intraday theo mã và theo phút
fact_market_index	toYYYYMM(trading_date)	(index_id, trading_date)	Tối ưu truy vấn lịch sử chỉ số thị trường

<code>fact_news_sentiment_daily</code>	<code>toYYYYMM(news_date)</code>	<code>(ticker, news_date)</code>	Tối ưu truy vấn sentiment theo mã và thời gian
<code>fact_alert_event</code>	<code>toYYYYMM(triggered_at)</code>	<code>(user_id, triggered_at)</code>	Tối ưu truy vấn lịch sử cảnh báo theo người dùng

3.3.6. Lý do lựa chọn thiết kế này

Thiết kế Star Schema trên phù hợp với hệ thống Data Lakehouse theo dõi thị trường chứng khoán vì các lý do sau:

Thứ nhất, mô hình này tách biệt rõ giữa dữ liệu mô tả và dữ liệu đo lường. Các bảng Dimension như `dim_stock`, `dim_sector`, `dim_date`, `dim_index` cung cấp ngữ cảnh phân tích, trong khi các bảng Fact lưu dữ liệu giao dịch, chỉ báo kỹ thuật, dữ liệu realtime, dữ liệu sentiment và lịch sử cảnh báo.

Thứ hai, schema được thiết kế trực tiếp theo các nhu cầu dashboard.

`fact_daily_price` phục vụ phân tích kỹ thuật, biểu đồ giá và xếp hạng cổ phiếu; `fact_market_index` phục vụ dashboard tổng quan thị trường; `fact_realtime_vwap` phục vụ giám sát intraday và cảnh báo realtime; `fact_news_sentiment_daily` phục vụ phân tích tác động tin tức; `fact_alert_event` phục vụ dashboard lịch sử cảnh báo.

Thứ ba, thiết kế này phù hợp với ClickHouse. Các bảng fact có thể được partition theo ngày hoặc theo tháng tùy theo granularity dữ liệu. Các khóa `ORDER BY` được lựa chọn dựa trên mẫu truy vấn phổ biến của dashboard như xem lịch sử giá theo mã, theo dõi realtime theo phút hoặc xem lịch sử cảnh báo theo người dùng.

Thứ tư, schema hỗ trợ tốt cả batch analytics và realtime analytics. Dữ liệu OHLCV cuối ngày được xử lý qua pipeline batch, trong khi dữ liệu VWAP và cảnh báo realtime được ghi nhận theo từng phút hoặc từng sự kiện, giúp hệ thống đáp ứng cả nhu cầu phân tích lịch sử và theo dõi thị trường trong phiên.

Thứ năm, schema có khả năng mở rộng. Trong tương lai, hệ thống có thể bổ sung thêm các bảng như `fact_financial_metric` để lưu chỉ số tài chính theo quý, `fact_user_watchlist` để phân tích hành vi người dùng, hoặc `fact_news_article` để lưu chi tiết từng bài viết.

IV. Chi Tiết Luồng Xử Lý ETL Từng Loại Dữ Liệu

Hệ thống được thiết kế với 5 luồng xử lý (pipeline) chuyên biệt, tương ứng với đặc thù của từng loại dữ liệu chứng khoán:

1. Dữ liệu Giá Cổ Phiếu (OHLCV — Batch hàng ngày)

- **Thu thập (Ingestion):** Apache Airflow lên lịch gọi API (vnstock) hàng ngày để lấy toàn bộ mã HOSE/HNX. Dữ liệu thô lưu vào Bronze (MinIO, Parquet).
- **Làm sạch — Silver Layer:** Polars + Great Expectations kiểm tra lỗi logic (giá âm, volume = 0, high < low, trùng ngày). Cast kiểu string → float/date, điền null bằng công thức hợp lý. Kết quả lưu vào Silver (MinIO).
- **Tính chỉ số kỹ thuật — Gold Layer (dbt):** dbt models trong ClickHouse tính toán: SMA-20, EMA-12/26, MACD, RSI-14, Bollinger Bands (20, $\pm 2\sigma$) trên dữ liệu Silver. Kết quả ghi vào bảng `fact_daily_price` — Gold Layer.
- **Phục vụ (Serving):** `fact_daily_price` là bảng truy vấn chính phục vụ vẽ biểu đồ nền và đưa tín hiệu mua/bán trên Superset.

2. Dữ liệu Số Lệnh (Streaming Real-time)

- **Thu thập:** DNSE WebSocket Producer đẩy từng lệnh khớp vào Kafka topic `raw_trades`.
- **Xử lý (Serverless):** ClickHouse Kafka Engine đọc trực tiếp topic `raw_trades`, Materialized View (AggregatingMergeTree) tự động tổng hợp theo từng phút: tính tổng `price × volume` và tổng volume cho mỗi mã.
- **VWAP Window:** VWAP trượt 5 phút được tính bằng cách query trên 5 bản ghi phút gần nhất từ bảng MV, hoặc dùng `windowView` (ClickHouse 23.4+) nếu cần độ trễ thấp hơn.
- **Alert Engine:** Consumer Python nhẹ query từ bảng MV, so sánh với ngưỡng Bollinger Band/RSI, gửi cảnh báo qua Telegram/Email.

3. Dữ Liệu Tin Tức Thị Trường (Phi cấu trúc)

- **Thu thập:** Scrapy crawl HTML/RSS hàng ngày, lưu thô dạng JSON vào Bronze (tiêu đề, nội dung, URL, ngày đăng, nguồn).
- **Làm sạch — Silver:** Polars loại bài trùng (URL + ngày), bỏ bài <50 chữ, chuẩn hóa datetime, dọn HTML tags. Lưu vào Silver.
- **Gán nhãn (Entity Linking):** Từ điển khóa ("Vietcombank" → VCB) + Regex quét tiêu đề gán mã cổ phiếu cho từng bài.
- **Phân tích NLP (Proof of Concept):** Gọi Claude/GPT API theo batch cuối ngày (~50 tin/ngày) để phân tích cảm xúc → POSITIVE/NEGATIVE/NEUTRAL + điểm tin cậy. (Phương án đơn giản, không cần self-host PhoBERT)
- **Tổng hợp — Gold:** Group by ticker + date: đếm số tin, tính avg_sentiment_score, lưu vào fact_news_sentiment_daily — dữ liệu này feed cho Text-to-SQL demo và dashboard Superset.

4. Thông Tin Doanh Nghiệp Niêm Yết (Batch hàng quý)

- **Thu thập:** Airflow gọi API hàng quý lấy: tên công ty, sàn, ngành, vốn hoá, P/E, EPS, ROE, ROA, ngày niêm yết, số cổ phiếu lưu hành.
- **Chuẩn hóa danh mục:** Quy chuẩn tên ngành về hệ thống GICS. Xử lý mã mới/hủy niêm yết/đổi tên.
- **Lưu trữ Dimension:** Upsert vào dim_stock (update nếu đã có, insert nếu chưa) theo primary key ticker. Bảng này là FK chính cho tất cả bảng fact.

5. Chỉ Số Thị Trường (Market Index)

- **Thu thập:** Lấy VN-Index, HNX-Index, VN30 mỗi ngày (điểm mở, cao, thấp, đóng, tổng khối lượng, tổng giá trị giao dịch), lưu Bronze.
- **Tính toán (Market Breadth) — Gold:** dbt tính SMA, EMA, RSI cho Index. Bổ sung: tỷ lệ tăng/giảm/đứng giá (advance/decline ratio) đo lường độ rộng thị trường.
- **Lưu trữ:** Lưu vào fact_market_index phục vụ dashboard tổng quan thị trường.

V. Lựa Chọn Công Nghệ Và Giải Trình Kiến Trúc

Việc lựa chọn công nghệ được đánh giá dựa trên: hiệu năng cao, phù hợp quy mô (<100GB) và tối ưu tài nguyên.

1. Thu Thập Dữ Liệu (Ingestion)

- **Luồng Batch:** Apache Airflow + Python (requests) gọi API lịch sử vào cuối ngày. Hỗ trợ retry tự động khi fail, theo dõi trạng thái từng task rõ ràng.
- **Luồng Streaming:** Kafka Producer (Python) kết nối trực tiếp WebSocket DNSE đẩy lệnh khớp vào topic raw_trades. Không dùng Spark/Flink ở bước ingest để tránh overkill tài nguyên.

2. Lưu Trữ Thô (Bronze — Data Lake)

- **Lựa chọn:** MinIO (Object Storage tự host) + Apache Parquet. Parquet columnar format đọc nhanh hơn CSV ~10 lần, nén tốt. Thay thế HDFS cũ cần cluster nhiều node — phù hợp quy mô vừa và nhỏ.

3. Biến Đổi Dữ Liệu (Transform — ETL)

- **Bronze → Silver:** Polars (Python): làm sạch, cast kiểu, chuẩn hóa. Với 1,700 mã × 10 năm (~4 triệu dòng), Polars nhanh hơn Pandas 10–20 lần trên máy đơn. Không dùng PySpark vì dữ liệu chưa vượt giới hạn RAM máy đơn.
- **Silver → Gold:** dbt (SQL models chạy trong ClickHouse): tính chỉ số kỹ thuật, tổng hợp fact tables. Mang lại version control, tài liệu tự sinh, lineage graph chuẩn industry.

4. Kiểm Soát Chất Lượng Dữ Liệu (Data Quality)

- **Lớp 1 — Sau Ingestion:** Great Expectations (GX) chạy ngay sau mỗi bước ingest. Nếu fail, Airflow dừng pipeline và gửi alert. Kiểm tra: giá âm, volume null, high < low, trùng ngày.
- **Lớp 2 — Sau Transform:** dbt tests (not_null, unique, accepted_values) chạy tự động sau mỗi lần transform trong ClickHouse.

5. Xử Lý Streaming (Real-time)

- **Giải pháp Serverless:** ClickHouse Kafka Engine đọc trực tiếp Kafka topic như bảng ngoài. Materialized Views tự aggregate VWAP, tổng khối lượng, giá trung bình ngay khi insert. Alert Engine chỉ là consumer Python nhẹ đọc kết quả đã tính sẵn.
- **Ưu điểm:** Loại bỏ hoàn toàn lớp xử lý trung gian (Faust/Kafka Streams/Spark Streaming), giảm độ trễ mạng, tiết kiệm tài nguyên. Phù hợp với ~2–3 triệu tick/ngày của thị trường Việt Nam.

6. Phục Vụ Truy Vấn & Hiển Thị (BI)

- **Database chính:** ClickHouse (OLAP): nhận batch từ dbt, nhận streaming từ Kafka Engine, phục vụ truy vấn mili-giây.
- **Visualization:** Apache Superset: dashboard phân tích báo cáo (batch analytics). Grafana: giám sát real-time (VWAP, Kafka consumer lag, pipeline health).

VI. Thiết Kế Alert Engine & Quản Lý Ngưỡng Người Dùng

Alert Engine là module cho phép người dùng đặt điều kiện cảnh báo theo từng mã cổ phiếu, ví dụ: cảnh báo khi giá vượt một ngưỡng nhất định, RSI vượt 70, RSI dưới 30, giá vượt Bollinger Band hoặc giá lệch quá xa so với VWAP lũy kế phiên.

Module này được thiết kế tách riêng giữa hai loại dữ liệu:

- **Dữ liệu cấu hình cảnh báo:** lưu trong PostgreSQL, dùng để quản lý các ngưỡng cảnh báo do người dùng đặt.
- **Dữ liệu lịch sử cảnh báo đã kích hoạt:** lưu trong ClickHouse, dùng để truy vấn nhanh theo thời gian, hiển thị dashboard và hỗ trợ kiểm tra chống spam.

Cách tách này giúp hệ thống vừa đảm bảo tính linh hoạt khi người dùng thêm/sửa/xóa cảnh báo, vừa tối ưu hiệu năng phân tích lịch sử cảnh báo trên khối lượng dữ liệu lớn.

6.1. Kiến trúc Alert Engine

Alert Engine gồm ba thành phần chính:

Thành phần	Vai trò	Công nghệ
<code>user_alerts</code> table	Lưu cấu hình cảnh báo của từng người dùng	PostgreSQL
Alert Checker	Đọc dữ liệu mới nhất từ ClickHouse, so sánh với ngưỡng trong <code>user_alerts</code> , kiểm tra chống spam trước khi gửi	Python consumer chạy định kỳ mỗi 60 giây
Notification Sender	Gửi cảnh báo qua Telegram hoặc Email, sau đó ghi log cảnh báo vào <code>fact_alert_event</code>	Telegram Bot API / SMTP Email

fact_alert_event table Lưu lịch sử các cảnh báo đã kích hoạt và đã gửi ClickHouse

Trong kiến trúc này, PostgreSQL được dùng cho dữ liệu cấu hình vì bảng **user_alerts** có tính giao dịch cao, thường xuyên được thêm, sửa, bật/tắt bởi người dùng. Ngược lại, ClickHouse được dùng để lưu **fact_alert_event** vì đây là dữ liệu sự kiện tăng dần theo thời gian, cần truy vấn nhanh để hiển thị lịch sử cảnh báo, thống kê số lượng alert và kiểm tra các alert bị kích hoạt lặp lại.

6.2. Schema bảng **user_alerts** trong PostgreSQL

Bảng **user_alerts** lưu cấu hình cảnh báo do người dùng thiết lập. Mỗi dòng tương ứng với một điều kiện cảnh báo cho một mã cổ phiếu cụ thể.

Cột	Kiểu dữ liệu	Ví dụ giá trị	Mô tả
alert_id	UUID	uuid-1234	Khóa chính của cảnh báo
user_id	VARCHAR	user_001	ID người dùng đặt cảnh báo
ticker	VARCHAR(10)	VCB	Mã cổ phiếu cần theo dõi
condition_type	ENUM	RSI_ABOVE	Loại điều kiện cảnh báo
threshold_value	FLOAT	70	Ngưỡng kích hoạt cảnh báo
channel	ENUM	TELEGRAM	Kênh gửi cảnh báo: TELEGRAM hoặc EMAIL
is_active	BOOLEAN	true	Bật/tắt cảnh báo mà không cần xóa
cooldown_minutes	INT	30	Khoảng thời gian tối thiểu trước khi gửi lại cảnh báo cùng loại

<code>created_at</code>	TIMESTAMP	2024-01-15 09:00:00	Thời điểm tạo cảnh báo
<code>updated_at</code>	TIMESTAMP	2024-01-15 09:30:00	Thời điểm cập nhật cấu hình cảnh báo

Các giá trị phổ biến của `condition_type` gồm:

Giá trị	Ý nghĩa
<code>PRICE_ABOVE</code>	Cảnh báo khi giá lớn hơn một ngưỡng
<code>PRICE_BELOW</code>	Cảnh báo khi giá nhỏ hơn một ngưỡng
<code>RSI_ABOVE</code>	Cảnh báo khi RSI vượt một ngưỡng, ví dụ RSI > 70
<code>RSI_BELOW</code>	Cảnh báo khi RSI thấp hơn một ngưỡng, ví dụ RSI < 30
<code>BB_BREAK</code>	Cảnh báo khi giá vượt Bollinger Band trên hoặc dưới
<code>VWAP_DEVIATION</code>	Cảnh báo khi giá lệch quá xa so với session VWAP

Trường `cooldown_minutes` được dùng để cấu hình cơ chế chống spam. Giá trị mặc định là **30 phút**, nghĩa là sau khi một cảnh báo đã được gửi, hệ thống sẽ không gửi lại cảnh báo cùng `user_id + ticker + condition_type` trong vòng 30 phút tiếp theo.

6.3. Bảng `fact_alert_event` trong ClickHouse

Bảng `fact_alert_event` lưu lịch sử các cảnh báo đã được kích hoạt bởi Alert Engine. Bảng này không lưu cấu hình cảnh báo, mà lưu các sự kiện cảnh báo thực tế đã xảy ra.

Cột	Kiểu dữ liệu	Mô tả
<code>alert_id</code>	UUID	ID cảnh báo được kích hoạt
<code>user_id</code>	String	Người dùng nhận cảnh báo
<code>ticker</code>	String	Mã cổ phiếu liên quan
<code>date_id</code>	UInt32	Khóa ngày, liên kết với <code>dim_date</code>
<code>triggered_at</code>	DateTime	Thời điểm cảnh báo được kích hoạt
<code>condition_type</code>	LowCardinality(String)	Loại điều kiện cảnh báo
<code>threshold_value</code>	Float64	Ngưỡng cảnh báo người dùng đặt
<code>actual_value</code>	Float64	Giá trị thực tế tại thời điểm kích hoạt
<code>channel</code>	LowCardinality(String)	Kênh gửi cảnh báo: TELEGRAM hoặc EMAIL
<code>is_sent</code>	UInt8	Đánh dấu cảnh báo đã gửi thành công hay chưa
<code>sent_at</code>	Nullable(DateTime)	Thời điểm gửi cảnh báo
<code>created_at</code>	DateTime	Thời điểm ghi log cảnh báo

Bảng này được lưu trong ClickHouse để phục vụ các truy vấn phân tích nhanh như:

- Xem lịch sử cảnh báo của một người dùng.
- Thống kê số lượng cảnh báo theo ngày.
- Thống kê số lượng cảnh báo theo mã cổ phiếu.
- Kiểm tra tỷ lệ cảnh báo gửi thành công/thất bại.
- Phát hiện các cảnh báo bị kích hoạt lặp lại trong thời gian ngắn.

Gợi ý thiết kế vật lý trong ClickHouse:


```
ENGINE = MergeTree
PARTITION BY toYYYYMM(triggered_at)
ORDER BY (user_id, triggered_at)
```

Lý do lựa chọn:

- **PARTITION BY toYYYYMM(triggered_at)** giúp quản lý dữ liệu cảnh báo theo tháng, phù hợp với dữ liệu log/sự kiện tăng dần theo thời gian.
 - **ORDER BY (user_id, triggered_at)** tối ưu cho truy vấn phổ biến nhất: xem lịch sử cảnh báo của một người dùng cụ thể theo thời gian.
 - Các trường **condition_type** và **channel** sử dụng **LowCardinality(String)** vì đây là các trường có số lượng giá trị nhỏ, lặp lại nhiều lần, giúp ClickHouse tối ưu lưu trữ và tăng tốc truy vấn lọc/nhóm.
-

6.4. Quy trình hoạt động của Alert Engine

Quy trình xử lý cảnh báo được thực hiện theo các bước sau:

1. Người dùng tạo cảnh báo, ví dụ: “Gửi Telegram khi RSI của VCB vượt 70”.
2. Cấu hình cảnh báo được lưu vào bảng **user_alerts** trong PostgreSQL.
3. Alert Checker chạy định kỳ mỗi 60 giây.
4. Alert Checker đọc danh sách cảnh báo đang bật từ **user_alerts**, tức các bản ghi có **is_active = true**.
5. Alert Checker truy vấn dữ liệu mới nhất từ ClickHouse, ví dụ:
 - RSI từ **fact_daily_price**.
 - Giá hiện tại, VWAP 1 phút, session VWAP từ **fact_realtime_vwap**.
 - Bollinger Band từ **fact_daily_price**.
6. Alert Checker so sánh giá trị thực tế với ngưỡng người dùng đã đặt.
7. Nếu điều kiện chưa thỏa mãn, hệ thống không làm gì.
8. Nếu điều kiện thỏa mãn, hệ thống kiểm tra cơ chế chống spam.
9. Nếu không vi phạm cooldown, Notification Sender gửi cảnh báo qua Telegram hoặc Email.
10. Sau khi gửi, hệ thống ghi một bản ghi mới vào **fact_alert_event**.

Ví dụ:

Người dùng đặt cảnh báo:

ticker = VCB

condition_type = RSI_ABOVE

threshold_value = 70

channel = TELEGRAM

cooldown_minutes = 30

Khi Alert Checker chạy:

RSI hiện tại của VCB = 72

Điều kiện RSI_ABOVE được kích hoạt.

Hệ thống kiểm tra xem cảnh báo RSI_ABOVE cho user này và mã VCB đã được gửi trong 30 phút gần nhất chưa.

Nếu chưa có, hệ thống gửi Telegram và ghi log vào fact_alert_event.

Nếu đã có, hệ thống bỏ qua lần kích hoạt này để tránh spam.

6.5. Cơ chế chống spam cảnh báo

Cơ chế chống spam được thực thi tại **Alert Checker**, trước khi hệ thống gửi cảnh báo qua Telegram hoặc Email. Dashboard chỉ có vai trò hiển thị lại lịch sử cảnh báo, không phải nơi thực thi chống spam.

Khi một điều kiện cảnh báo được kích hoạt, Alert Checker sẽ query bảng **fact_alert_event** trong ClickHouse để kiểm tra xem cùng một tổ hợp:

user_id + ticker + condition_type

đã được gửi trong vòng N phút gần nhất hay chưa. Giá trị N được lấy từ trường **cooldown_minutes** trong bảng **user_alerts**, mặc định là **30 phút**.

Nếu đã tồn tại một cảnh báo giống nhau trong khoảng thời gian N phút gần nhất, hệ thống bỏ qua lần kích hoạt hiện tại và không gửi thông báo. Nếu chưa tồn tại, hệ thống gửi cảnh báo và ghi một bản ghi mới vào **fact_alert_event**.

Ví dụ logic chống spam:

```
SELECT count()  
FROM fact_alert_event  
WHERE user_id = 'user_001'
```

```
AND ticker = 'VCB'
AND condition_type = 'RSI_ABOVE'
AND is_sent = 1
AND triggered_at >= now() - INTERVAL 30 MINUTE;
```

Nếu kết quả lớn hơn 0, hệ thống không gửi lại cảnh báo. Nếu kết quả bằng 0, hệ thống gửi cảnh báo mới.

Cơ chế này giúp tránh trường hợp một mã cổ phiếu dao động quanh ngưỡng cảnh báo và liên tục kích hoạt thông báo trong thời gian ngắn. Ví dụ, nếu RSI của VCB dao động quanh mức 70, hệ thống sẽ không gửi Telegram liên tục mỗi 60 giây mà chỉ gửi lại sau khi hết khoảng cooldown.

6.6. Phân biệt vai trò giữa **user_alerts**, **fact_alert_event** và Dashboard

Thành phần	Nơi lưu/hiển thị	Vai trò
user_alerts	PostgreSQL	Lưu cấu hình cảnh báo của người dùng, gồm mã cổ phiếu, loại điều kiện, ngưỡng, kênh gửi và cooldown
Alert Checker	Python consumer	Kiểm tra điều kiện cảnh báo, thực thi chống spam và quyết định có gửi cảnh báo hay không
Notification Sender	Telegram Bot API / SMTP Email	Gửi cảnh báo đến người dùng
fact_alert_event	ClickHouse	Lưu lịch sử cảnh báo đã kích hoạt và đã gửi
Alert History Dashboard	Superset	Hiển thị lại lịch sử cảnh báo, thống kê alert và hỗ trợ kiểm tra các cảnh báo bị kích hoạt lặp

Như vậy, cần phân biệt rõ:

- **Bộ lọc trên dashboard** dùng để xem lại lịch sử cảnh báo và phát hiện alert từng bị kích hoạt sát nhau.
- **Cơ chế chống spam thật sự** được thực hiện trong Alert Checker trước khi gửi thông báo.

- **Bảng `fact_alert_event`** vừa là nơi lưu log lịch sử, vừa là nguồn để Alert Checker kiểm tra lần gửi gần nhất.

Thiết kế này giúp hệ thống vừa tránh làm phiền người dùng bởi các cảnh báo trùng lặp, vừa giữ lại đầy đủ lịch sử cảnh báo để phục vụ dashboard, phân tích và kiểm tra sau này.

VII. Các Điểm Nhấn Kỹ Thuật Nâng Cao (Engineering Highlights)

Dự án triển khai 3 giải pháp kỹ thuật chuyên sâu nhằm đảm bảo hệ thống vận hành ổn định, dễ bảo trì và chịu lỗi tốt:

1. Serverless Streaming — Xử Lý Không Cần Server Trung Gian

Vấn đề: Hầu hết luồng streaming cần nuôi một cụm server (Faust/Spark) trung gian để đọc Kafka, tính VWAP, rồi mới đẩy vào CSDL — gây công kênh và tốn tài nguyên.

Giải pháp: ClickHouse Kafka Engine + Materialized Views (AggregatingMergeTree) tự động aggregate streaming ngay khi data vào — loại bỏ hoàn toàn lớp xử lý trung gian.

Giá trị: Kiến trúc tinh gọn (Lean Architecture), giảm bottleneck, chứng minh hiệu suất mạnh OLAP Database hiện đại.

2. DataOps & CI/CD cho dbt + Airflow

Vấn đề: Sửa logic tính toán thủ công dễ làm chết pipeline hàng ngày.

Giải pháp: GitHub Actions tự động tạo schema test trong ClickHouse khi push code dbt/Airflow mới. Chỉ merge vào Production khi pass toàn bộ dbt tests + GX checks.

Giá trị: Tự duy DataOps chuẩn industry, bảo vệ hệ thống khỏi code lỗi làm hỏng dữ liệu.

3. Data Replay — Tự Động Vá Lỗ Đứt Gãy Dữ Liệu

Vấn đề: Mạng rớt 5 phút trong giờ giao dịch → một số tick mất → VWAP theo phút bị sai lệch. Lưu ý quan trọng: API nguồn chỉ cấp dữ liệu OHLCV theo ngày, không cấp tick theo từng phút — không thể dùng OHLCV ngày để phục hồi VWAP phút bị mất.

Giải pháp: Song song với Kafka Consumer, toàn bộ raw tick từ WebSocket được lưu vào MinIO Bronze (dạng JSON/Parquet, phân vùng theo ngày). Cuối ngày, một job batch Airflow đọc lại toàn bộ tick từ file Bronze, tính lại VWAP chính xác cho từng phút, sau đó ghi đè vào bảng `fact_realtime_vwap`. Đảm bảo dữ liệu cuối ngày luôn đúng ngay cả khi streaming bị gián đoạn.

Giá trị: Fault Tolerance chuyên nghiệp, hệ thống tự vá lỗi dựa trên dữ liệu thực (raw tick đã lưu), không phụ thuộc vào nguồn API ngoài sau sự cố.

VIII. Bảo Mật Dữ Liệu (Data Security)

Do đây là lĩnh vực tài chính nhạy cảm, hệ thống triển khai các biện pháp bảo mật sau:

Biện pháp	Mô tả	Công nghệ
Row-Level Security (RLS)	Người dùng chỉ xem được danh mục cổ phiếu được phân quyền — cân bằng user_stock_permission (user_id, ticker). Trong phạm vi đồ án, RLS được thiết kế ở mức demo với dữ liệu mẫu. Để áp dụng thực tế cần tích hợp thêm hệ thống đăng nhập và bảng phân quyền người dùng.	ClickHouse Row Policy + Superset RLS (sẵn sàng mở rộng)
API Key Management	Không hardcode key trong code. Quản lý qua biến môi trường hoặc secret manager	Airflow Variables/Connections, .env
Network Isolation	ClickHouse, Kafka, MinIO không expose public. Chỉ truy cập qua internal network	Docker Network, Firewall rules
Data Encryption	Mã hóa dữ liệu ở trạng thái lưu trữ (at rest) và truyền tải (in transit)	MinIO TLS, Kafka SSL/TLS, HTTPS

IX.

IX. Thiết kế Dashboard cho hệ thống

Dashboard của hệ thống được thiết kế nhằm giúp người dùng theo dõi thị trường chứng khoán từ mức tổng quan đến chi tiết từng mã cổ phiếu. Các trang dashboard được chia theo từng nhóm nhu cầu chính: tổng quan thị trường, phân tích mã cổ phiếu, quét tín hiệu kỹ thuật, giám sát chất lượng dữ liệu, theo dõi realtime VWAP, phân tích tin tức/sentiment và lịch sử cảnh báo.

Hệ thống sử dụng **Apache Superset** cho các dashboard phân tích phục vụ nhà đầu tư, sử dụng **Grafana** để giám sát realtime metrics và trạng thái service, đồng thời sử dụng **Airflow UI** để theo dõi trạng thái DAG và task trong pipeline batch.

1. Trang Market Overview — Tổng quan thị trường

Mục đích

Trang Market Overview cung cấp cái nhìn nhanh về trạng thái chung của thị trường trong ngày. Người dùng có thể biết thị trường đang tăng hay giảm, thanh khoản ra sao, chỉ số chính biến động thế nào và nhóm ngành nào đang dẫn dắt thị trường.

Nguồn dữ liệu chính

- fact_market_index
- fact_daily_price
- dim_stock
- dim_sector
- dim_date

Bộ lọc chính

Bộ lọc	Mô tả
Ngày giao dịch	Mặc định hiển thị phiên giao dịch gần nhất
Sàn giao dịch	HOSE, HNX, UPCOM
Ngành	Lọc theo nhóm ngành
Chỉ số	VN-Index, HNX-Index, VN30

Thông tin hiển thị

Nhóm thông tin	Nội dung
Chỉ số thị trường	VN-Index, HNX-Index, VN30

Biến động thị trường	Điểm tăng/giảm, % thay đổi so với phiên trước
Thanh khoản	Tổng khối lượng giao dịch, tổng giá trị giao dịch
Độ rộng thị trường	Số mã tăng, số mã giảm, số mã đứng giá
Ngành dẫn dắt	Ngành tăng mạnh nhất, ngành giảm mạnh nhất
Top cổ phiếu	Top tăng giá, top giảm giá, top thanh khoản, top vốn hóa

Biểu đồ nên dùng

Biểu đồ	Mục đích
KPI Card	Hiển thị VN-Index, % thay đổi, tổng giá trị giao dịch
Line Chart	Diễn biến VN-Index/VN30 theo thời gian
Bar Chart	Top 10 mã tăng mạnh nhất và top 10 mã giảm mạnh nhất
Horizontal Bar Chart	Top 10 mã có khối lượng hoặc giá trị giao dịch cao nhất
Donut Chart	Tỷ lệ số mã tăng, giảm, đứng giá
Heatmap	Hiệu suất theo ngành

Liên kết drill-down

Người dùng có thể nhấn vào mã cổ phiếu trong bảng Top tăng giá, Top giảm giá, Top thanh khoản hoặc Top vốn hóa để chuyển sang trang **Stock Detail** với mã cổ phiếu đó được chọn tự động.

Ví dụ insight người dùng nhận được

- VN-Index tăng nhưng số mã giảm nhiều hơn số mã tăng, cho thấy thị trường phân hóa.
- Ngành Ngân hàng đang dẫn dắt thị trường.

- Một số mã có thanh khoản tăng đột biến so với trung bình 20 ngày.
- Top vốn hóa giúp người dùng nhận biết các cổ phiếu có ảnh hưởng lớn đến chỉ số chung.

2. Trang Stock Detail — Phân tích chi tiết mã cổ phiếu

Mục đích

Trang Stock Detail cho phép người dùng chọn một mã cổ phiếu cụ thể để xem lịch sử giá, thanh khoản, vốn hóa, chỉ báo kỹ thuật và tin tức liên quan. Đây là trang quan trọng nhất đối với người dùng muốn phân tích sâu một cổ phiếu.

Nguồn dữ liệu chính

- `fact_daily_price`
- `dim_stock`
- `dim_sector`
- `fact_news_sentiment_daily`
- `dim_date`

Bộ lọc chính

Bộ lọc	Mô tả
Mã cổ phiếu	Người dùng chọn một ticker cụ thể, ví dụ: VCB, FPT, HPG
Khoảng thời gian	1 tháng, 3 tháng, 6 tháng, 1 năm hoặc tùy chọn
Mặc định	Hiển thị 3 tháng gần nhất
Chỉ báo kỹ thuật	Bật/tắt SMA, EMA, Bollinger Band, RSI, MACD

Việc đặt mặc định 3 tháng gần nhất giúp dashboard hiển thị đủ xu hướng ngắn và trung hạn mà không làm biểu đồ quá dài hoặc khó đọc.

Thông tin hiển thị

Nhóm thông tin	Nội dung
----------------	----------

Thông tin mã	Ticker, tên công ty, sàn, ngành, trạng thái niêm yết
Giá giao dịch	Open, High, Low, Close, Volume, Value
Biến động giá	Price change, pct_change
Vốn hóa	Market cap theo ngày
Chỉ báo xu hướng	SMA20, EMA12, EMA26
Chỉ báo động lượng	RSI14, MACD, MACD Signal
Chỉ báo biến động	Bollinger Band
Tin tức	Số lượng tin, số nguồn tin, sentiment trung bình, headline nổi bật

Biểu đồ nên dùng

Biểu đồ	Mục đích
Candlestick Chart	Hiển thị biến động OHLC theo ngày
Line Chart	Hiển thị SMA20, EMA12, EMA26 trên giá đóng cửa
Bar Chart	Khối lượng giao dịch theo ngày
Line Chart	RSI14 theo thời gian, kèm ngưỡng 30 và 70
Line Chart	MACD và MACD Signal
Line/Band Chart	Bollinger Band và giá đóng cửa
KPI Card	Close price, pct_change, volume, market_cap
Table	Tin tức nổi bật và điểm sentiment

Liên kết drill-down

Từ các bảng tin tức, bảng tín hiệu kỹ thuật hoặc bảng xếp hạng cổ phiếu, người dùng có thể nhấn vào mã cổ phiếu để chuyển sang trang **Stock Detail** và xem chi tiết lịch sử giá, chỉ báo và tin tức liên quan.

Ví dụ insight người dùng nhận được

- Cổ phiếu VCB có RSI = 72, có thể đang rơi vào vùng quá mua.
- Giá đóng cửa vượt Bollinger Band trên, thể hiện biến động mạnh.
- Khối lượng hôm nay cao hơn trung bình 20 ngày, cho thấy dòng tiền tăng.
- Tin tức tích cực xuất hiện cùng giai đoạn giá tăng, giúp người dùng có thêm ngữ cảnh phân tích.

3. Trang Technical Signal Scanner — Bộ lọc tín hiệu kỹ thuật

Mục đích

Trang Technical Signal Scanner tự động quét toàn bộ thị trường để tìm các mã có tín hiệu kỹ thuật đáng chú ý. Trang này giúp người dùng không cần tự tính từng chỉ báo mà vẫn có thể nhanh chóng phát hiện các mã có dấu hiệu quá mua, quá bán, breakout, breakdown hoặc thanh khoản đột biến.

Nguồn dữ liệu chính

- `fact_daily_price`
- `dim_stock`
- `dim_sector`
- `dim_date`

Bộ lọc chính

Bộ lọc	Mô tả
Ngày giao dịch	Mặc định là phiên giao dịch gần nhất
Sàn giao dịch	HOSE, HNX, UPCOM
Ngành	Lọc theo nhóm ngành
Loại tín hiệu	RSI, MACD, Bollinger Band, Volume breakout

Khoảng giá trị RSI Cho phép lọc RSI theo khoảng tùy chọn

Khoảng thanh khoản Lọc theo volume hoặc value

Thông tin hiển thị

Nhóm tín hiệu	Điều kiện gợi ý
Quá mua	<code>rsi_14 > 70</code>
Quá bán	<code>rsi_14 < 30</code>
Breakout	<code>close > bb_upper</code>
Breakdown	<code>close < bb_lower</code>
MACD tích cực	<code>macd > macd_signal</code>
Thanh khoản đột biến	<code>volume > volume_sma_20 * 1.5</code>
Tăng giá mạnh	<code>pct_change</code> thuộc top cao nhất
Giảm giá mạnh	<code>pct_change</code> thuộc top thấp nhất

Ngưỡng thanh khoản đột biến được đặt là `volume > volume_sma_20 * 1.5`. Ngưỡng 1.5 lần trung bình 20 ngày được chọn vì đủ để lọc ra các mã có thanh khoản tăng đáng kể, nhưng không quá khắt khe như ngưỡng 2 lần khiến số lượng mã phát hiện được quá ít. Trong phạm vi demo, ngưỡng này có thể cấu hình lại tùy theo nhu cầu phân tích.

Biểu đồ nên dùng

Biểu đồ

Mục đích

KPI Card Số mã quá mua, quá bán, breakout, breakdown

Filter Danh sách mã thỏa mãn từng tín hiệu
Table

Bar Chart Top mã có RSI cao nhất

Bar Chart Top mã có RSI thấp nhất

Bar Chart Top mã có volume tăng đột biến

Table Danh sách mã có MACD tích cực hoặc giá vượt
Bollinger Band

Không sử dụng Scatter Plot cho toàn bộ thị trường vì số lượng mã lớn có thể khiến biểu đồ khó đọc và khó demo. Thay vào đó, hệ thống ưu tiên bảng lọc và biểu đồ xếp hạng để người dùng dễ xác định mã cần quan tâm.

Liên kết drill-down

Người dùng có thể nhấn vào mã cổ phiếu trong bảng tín hiệu để chuyển sang trang **Stock Detail** và xem diễn biến giá, RSI, MACD, Bollinger Band của mã đó trong khoảng thời gian gần nhất.

Ví dụ insight người dùng nhận được

- Có nhiều mã đang ở vùng quá mua, cho thấy thị trường có thể đang nóng.
- Một số mã vượt Bollinger Band trên kèm thanh khoản cao, có thể đang có dòng tiền mạnh.
- Các mã RSI thấp có thể được đưa vào danh sách theo dõi để tìm cơ hội hồi phục.

4. Trang Data Pipeline Monitor — Giám sát pipeline và chất lượng dữ liệu

Mục đích

Trang Data Pipeline Monitor dành cho admin, developer hoặc analyst nội bộ để theo dõi trạng thái pipeline thu thập, xử lý và kiểm soát chất lượng dữ liệu. Đây là điểm khác biệt quan trọng so với các ứng dụng chứng khoán thông thường, vì hệ thống không chỉ hiển thị dữ liệu mà còn giám sát độ tin cậy của dữ liệu.

Ghi chú về công cụ hiển thị

Trang Data Pipeline Monitor là trang tổng hợp các KPI và đường dẫn giám sát từ nhiều công cụ khác nhau.

- Phần trạng thái DAG, lịch chạy và task lỗi được hiển thị qua **Airflow UI**.
- Phần streaming health, Kafka consumer lag, trạng thái service và các chỉ số vận hành realtime được hiển thị qua **Grafana**.
- Phần data quality có thể lấy từ **Great Expectations validation results**, **dbt test results** và hiển thị lại trên Superset hoặc trang giám sát nội bộ.
- Trang này đóng vai trò như một trang tổng hợp, giúp người vận hành truy cập nhanh các thông tin quan trọng thay vì phải mở từng công cụ riêng lẻ.

Nguồn dữ liệu chính

- Airflow task logs
- Great Expectations validation results
- dbt test results
- Kafka consumer lag
- ClickHouse system tables
- MinIO object metadata
- Grafana metrics

Thông tin hiển thị

Nhóm thông tin	Công cụ hiển thị chính	Nội dung
Trạng thái pipeline batch	Airflow UI	DAG success/fail, thời gian chạy gần nhất, task bị lỗi
Ingestion volume	Airflow UI / Superset	Số bản ghi ingest theo ngày, số file ghi vào Bronze/Silver
Data quality	Great Expectations / dbt / Superset	Số lỗi GX validation, số test dbt pass/fail
Streaming health	Grafana	Kafka consumer lag, số message xử lý mỗi phút
Storage health	Grafana / MinIO Console	Dung lượng MinIO, số object theo từng layer Bronze/Silver

Serving health	Grafana / ClickHouse system tables	Trạng thái ClickHouse, thời gian query trung bình
----------------	------------------------------------	---

Biểu đồ nên dùng

Biểu đồ	Mục đích
KPI Card	Số DAG thành công/thất bại, số lỗi validation
Line Chart	Số bản ghi ingest theo ngày
Bar Chart	Số lỗi data quality theo loại lỗi
Table	Danh sách task Airflow thất bại gần nhất
Line Chart	Kafka consumer lag theo thời gian
Gauge/Stat Panel	Trạng thái service: Kafka, ClickHouse, MinIO, Airflow

Ví dụ insight admin nhận được

- Pipeline OHLCV hôm nay chạy thành công nhưng có một số mã bị thiếu dữ liệu.
- Great Expectations phát hiện lỗi **high < low** ở một số bản ghi.
- dbt test fail do có bản ghi trùng khóa (**ticker, date_id**).
- Kafka consumer lag tăng bất thường, cần kiểm tra streaming consumer.
- ClickHouse query dashboard vẫn nằm trong ngưỡng chấp nhận được.

5. Trang Realtime VWAP Monitoring — Theo dõi realtime trong phiên

Mục đích

Trang Realtime VWAP Monitoring theo dõi biến động intraday của cổ phiếu theo từng phút, đặc biệt là so sánh giá hiện tại với VWAP theo phút và VWAP lũy kế từ đầu phiên. Trang này phục vụ nhu cầu giám sát thị trường trong phiên và hỗ trợ Alert Engine phát hiện các biến động bất thường.

Nguồn dữ liệu chính

- **fact_realtime_vwap**

- `dim_stock`
- `fact_alert_event`

Thời gian hoạt động

Dashboard này hiển thị dữ liệu realtime trong giờ giao dịch, từ khoảng **9:00 đến 14:45**. Ngoài giờ giao dịch, dashboard hiển thị dữ liệu của phiên giao dịch gần nhất thay vì dữ liệu live.

Trong trường hợp nguồn dữ liệu realtime bị giới hạn hoặc API không ổn định, hệ thống có thể demo bằng dữ liệu giả lập từ lịch sử để chứng minh luồng Kafka → ClickHouse → Dashboard vẫn hoạt động đúng.

Bộ lọc chính

Bộ lọc	Mô tả
Mã cổ phiếu	Chọn ticker cần theo dõi
Ngày giao dịch	Mặc định là phiên hiện tại hoặc phiên gần nhất
Khoảng thời gian trong phiên	Sáng, chiều hoặc tùy chọn theo phút
Ngưỡng lệch VWAP	Ví dụ: 1%, 2%, 3%

Thông tin hiển thị

Nhóm thông tin	Nội dung
Giá realtime	Open, High, Low, Close theo từng phút
VWAP	VWAP 1 phút và session VWAP
Thanh khoản intraday	Volume từng phút, session volume
Độ lệch VWAP	price_vs_vwap_pct, price_vs_session_vwap_pct
Cảnh báo realtime	Số cảnh báo đã kích hoạt trong phiên

Biểu đồ nên dùng

Biểu đồ	Mục đích
Line Chart	Giá theo phút và session VWAP
Bar Chart	Khối lượng giao dịch từng phút
Line Chart	price_vs_session_vwap_pct theo thời gian
Table	Danh sách mã lệch VWAP mạnh nhất
KPI Card	Session volume, session VWAP, số alert trong phiên

Liên kết drill-down

Người dùng có thể nhấn vào mã cổ phiếu trong bảng lệch VWAP để chuyển sang trang **Stock Detail** hoặc xem lịch sử cảnh báo liên quan tại trang **Alert History**.

Ví dụ insight người dùng nhận được

- Giá hiện tại của VCB đang cao hơn session VWAP 1.8%.
- Thanh khoản tăng mạnh trong 10 phút gần nhất.
- Một số mã đang lệch khỏi VWAP quá ngưỡng và đã kích hoạt cảnh báo.

6. Trang News & Sentiment — Tin tức và cảm xúc thị trường

Mục đích

Trang News & Sentiment giúp người dùng theo dõi mức độ xuất hiện trên truyền thông và cảm xúc tin tức liên quan đến từng mã cổ phiếu. Trang này bổ sung góc nhìn dữ liệu phi cấu trúc vào hệ thống phân tích thị trường.

Nguồn dữ liệu chính

- fact_news_sentiment_daily
- dim_stock
- dim_sector

- **dim_date**

Bộ lọc chính

Bộ lọc	Mô tả
Ngày hoặc khoảng ngày	Chọn ngày cụ thể hoặc khoảng thời gian
Mã cổ phiếu	Lọc theo ticker
Ngành	Lọc theo nhóm ngành
Sentiment	Positive, Negative, Neutral
Số nguồn tin tối thiểu	Lọc các mã được nhiều nguồn đề cập

Thông tin hiển thị

Nhóm thông tin	Nội dung
Số lượng tin	news_count theo mã và theo ngày
Số nguồn tin	source_count
Sentiment	positive_count, negative_count, neutral_count
Điểm cảm xúc	avg_sentiment_score
Tin nổi bật	top_headline
So sánh theo ngành	Ngành có sentiment tích cực/tiêu cực nhất

Biểu đồ nên dùng

Biểu đồ	Mục đích
Bar Chart	Top mã có nhiều tin nhất
Stacked Bar Chart	Số tin tích cực, tiêu cực, trung lập
Line Chart	Sentiment score theo thời gian

	gian
Heatmap	Sentiment trung bình theo ngành
Table	Headline nổi bật theo từng mã

Liên kết drill-down

Người dùng có thể nhấn vào mã cổ phiếu trong bảng tin tức để chuyển sang trang **Stock Detail** và xem liệu biến động giá có cùng chiều với sentiment hay không.

Ví dụ insight người dùng nhận được

- Một mã có 10 tin trong ngày nhưng chỉ đến từ 2 nguồn, mức độ lan tỏa chưa cao.
- Một ngành có sentiment tiêu cực tăng mạnh trong vài ngày gần đây.
- Tin tức tích cực xuất hiện cùng thời điểm giá và thanh khoản tăng.

7. Trang Alert History — Lịch sử cảnh báo người dùng

Mục đích

Trang Alert History cho phép người dùng xem lại các cảnh báo đã được kích hoạt, điều kiện kích hoạt, giá trị thực tế và trạng thái gửi cảnh báo. Trang này cũng giúp admin kiểm tra hệ thống có từng gửi cảnh báo lặp hoặc spam hay không.

Cần phân biệt rõ: **bộ lọc trên dashboard chỉ dùng để xem lại lịch sử cảnh báo**, còn **logic chống spam được thực thi trong Alert Checker trước khi cảnh báo được gửi đi**.

Nguồn dữ liệu chính

- `fact_alert_event`
- `dim_stock`
- `dim_date`

Bộ lọc chính

Bộ lọc	Mô tả
Người dùng	Lọc theo user_id
Mã cổ phiếu	Lọc theo ticker
Loại cảnh báo	RSI_ABOVE, RSI_BELOW, BB_BREAK, PRICE_BELOW, PRICE_ABOVE, VWAP_DEVIATION
Kênh gửi	TELEGRAM, EMAIL
Trạng thái gửi	Đã gửi, chưa gửi, lỗi gửi
Khoảng thời gian	Hôm nay, 7 ngày gần nhất, 30 ngày gần nhất hoặc tùy chọn
Khoảng cách tối thiểu giữa các alert	Lọc các cảnh báo trùng điều kiện trong vòng 30/60 phút để kiểm tra lịch sử có xuất hiện cảnh báo lặp hay không

Bộ lọc chống spam trên dashboard giúp người dùng và admin xem lại lịch sử để phát hiện các cảnh báo bị kích hoạt quá sát nhau. Tuy nhiên, bộ lọc này không phải là nơi ngăn gửi cảnh báo. Việc ngăn gửi cảnh báo trùng lặp được thực hiện ở tầng xử lý Alert Engine.

Cơ chế chống spam trong Alert Engine

Cơ chế chống spam được thực thi tại **Alert Checker** trước khi hệ thống gửi cảnh báo qua Telegram hoặc Email.

Quy trình xử lý như sau:

1. Alert Checker đọc cấu hình cảnh báo đang bật từ bảng **user_alerts** trong PostgreSQL.
2. Alert Checker truy vấn dữ liệu mới nhất từ ClickHouse, ví dụ RSI, Bollinger Band, giá hiện tại hoặc độ lệch so với session VWAP.
3. Khi điều kiện cảnh báo được kích hoạt, hệ thống chưa gửi ngay mà kiểm tra bảng **fact_alert_event** trong ClickHouse.
4. Hệ thống kiểm tra xem cùng một tổ hợp **user_id + ticker +**

`condition_type` đã được gửi trong vòng N phút gần nhất hay chưa.

5. Nếu đã có cảnh báo tương tự trong khoảng thời gian N phút, hệ thống bỏ qua lần kích hoạt hiện tại để tránh spam.
6. Nếu chưa có cảnh báo tương tự, hệ thống gửi thông báo qua Telegram hoặc Email, sau đó ghi một bản ghi mới vào `fact_alert_event`.

Giá trị N mặc định là **30 phút** và có thể cấu hình trong bảng `user_alerts`.

Ví dụ, người dùng có thể đặt cooldown 30 phút hoặc 60 phút tùy mức độ nhạy cảm của cảnh báo.

Ví dụ logic kiểm tra chống spam:

Kiểm tra trước khi gửi alert:

```
user_id = user_001
ticker = VCB
condition_type = RSI_ABOVE
cooldown_minutes = 30
```

Nếu đã tồn tại bản ghi trong `fact_alert_event` với cùng `user_id`, `ticker`, `condition_type` và `triggered_at >= now() - 30 phút` thì không gửi lại cảnh báo.

Ngược lại, gửi cảnh báo và ghi log vào `fact_alert_event`.

Về mặt kỹ thuật, cơ chế này giúp tách biệt rõ hai phần:

Thành phần	Vai trò
<code>user_alerts</code> trong PostgreSQL	Lưu cấu hình cảnh báo của người dùng: mã cổ phiếu, điều kiện, ngưỡng, kênh gửi, cooldown
<code>fact_alert_event</code> trong ClickHouse	Lưu lịch sử các cảnh báo đã kích hoạt và đã gửi
Alert Checker	Thực thi logic kiểm tra điều kiện và chống spam trước khi gửi cảnh báo

Alert History Dashboard Hiện thị lại lịch sử cảnh báo và hỗ trợ kiểm tra cảnh báo trùng lặp

Cách thiết kế này giúp hệ thống vừa tránh làm phiền người dùng bởi các cảnh báo lặp lại, vừa vẫn giữ đầy đủ lịch sử cảnh báo để phục vụ kiểm tra và phân tích sau này.

Thông tin hiển thị

Nhóm thông tin	Nội dung
Lịch sử cảnh báo	Thời điểm kích hoạt, mã cổ phiếu, loại điều kiện
Ngưỡng cảnh báo	threshold_value
Giá trị thực tế	actual_value
Kênh gửi	TELEGRAM hoặc EMAIL
Trạng thái gửi	is_sent, sent_at
Cooldown	Thời gian tối thiểu trước khi gửi lại cảnh báo cùng loại
Thông kê alert	Số alert theo ngày, theo mã, theo loại điều kiện

Biểu đồ nên dùng

Biểu đồ	Mục đích
Table	Lịch sử cảnh báo chi tiết
Bar Chart	Số cảnh báo theo ngày
Donut Chart	Tỷ lệ cảnh báo theo condition_type
Bar Chart	Top mã kích hoạt nhiều cảnh báo nhất
KPI Card	Tổng số alert, số alert đã gửi, số alert lỗi
Table	Danh sách cảnh báo có dấu hiệu bị kích hoạt lặp trong thời

gian ngắn

Liên kết drill-down

Người dùng có thể nhấn vào mã cổ phiếu trong bảng lịch sử cảnh báo để chuyển sang trang **Stock Detail** hoặc trang **Realtime VWAP Monitoring** nếu cảnh báo liên quan đến VWAP.

Ví dụ insight người dùng nhận được

- Người dùng nhận nhiều cảnh báo nhất ở nhóm RSI_ABOVE.
- Một mã cổ phiếu liên tục kích hoạt cảnh báo VWAP_DEVIATION trong phiên.
- Có thể kiểm tra cảnh báo đã gửi thành công qua Telegram hoặc Email chưa.
- Có thể phát hiện hệ thống từng kích hoạt nhiều cảnh báo giống nhau trong khoảng thời gian ngắn.

Cơ chế điều hướng giữa các trang

Dashboard được thiết kế theo hướng **drill-down**, giúp người dùng đi từ bức tranh tổng quan đến phân tích chi tiết.

Từ trang	Hành động	Chuyển đến
Market Overview	Nhấn vào mã trong Top tăng/giảm/thanh khoản/vốn hóa	Stock Detail
Technical Signal Scanner	Nhấn vào mã có tín hiệu kỹ thuật	Stock Detail
Realtime VWAP Monitoring	Nhấn vào mã lệch VWAP mạnh	Stock Detail hoặc Alert History
News & Sentiment	Nhấn vào mã có tin nổi bật	Stock Detail
Alert History	Nhấn vào mã trong lịch sử cảnh báo	Stock Detail

Data Pipeline Monitor

Nhấn vào pipeline lỗi

Airflow UI hoặc Grafana chi tiết

Cơ chế drill-down giúp người dùng không chỉ xem dữ liệu tổng quan mà còn nhanh chóng truy vết đến nguyên nhân hoặc chi tiết của từng mã cổ phiếu.

Thứ tự ưu tiên triển khai Dashboard

Nếu thời gian triển khai có hạn, nên ưu tiên theo thứ tự sau:

Mức độ ưu tiên	Trang dashboard	Lý do
Bắt buộc	Market Overview	Cho thấy toàn cảnh thị trường và giá trị tổng quan của hệ thống
Bắt buộc	Stock Detail	Thể hiện rõ năng lực phân tích kỹ thuật của hệ thống
Bắt buộc	Technical Signal Scanner	Chứng minh hệ thống không chỉ lưu dữ liệu mà còn tạo insight
Bắt buộc	Data Pipeline Monitor	Thể hiện điểm khác biệt của kiến trúc Data Lakehouse: dữ liệu có kiểm soát chất lượng, có giám sát pipeline và có khả năng phát hiện lỗi
Nên có	Realtime VWAP Monitoring	Thể hiện năng lực streaming và realtime analytics, nhưng phụ thuộc vào độ ổn định của nguồn dữ liệu realtime
Nâng cao	News & Sentiment	Bổ sung dữ liệu phi cấu trúc và yếu tố truyền thông
Nâng cao	Alert History	Hoàn thiện use case cảnh báo cá nhân hóa

Trong phạm vi đề án, bốn trang quan trọng nhất cần hoàn thiện là **Market Overview**, **Stock Detail**, **Technical Signal Scanner** và **Data Pipeline Monitor**. Đây là nhóm dashboard thể hiện rõ nhất giá trị cốt lõi của hệ thống:

thu thập dữ liệu, chuẩn hóa dữ liệu, phân tích kỹ thuật và kiểm soát chất lượng dữ liệu.

Trang **Realtime VWAP Monitoring** nên được triển khai nếu pipeline streaming hoạt động ổn định. Trong trường hợp API realtime bị giới hạn hoặc không ổn định, hệ thống có thể demo bằng dữ liệu giả lập từ lịch sử để chứng minh luồng Kafka → ClickHouse → Dashboard vẫn hoạt động đúng.

Timeline triển khai dự án trong 4 tuần

Dự án được triển khai trong 4 tuần với mục tiêu xây dựng hệ thống Data Lakehouse theo dõi thị trường chứng khoán. Hệ thống xử lý 5 luồng dữ liệu chính gồm: dữ liệu giá cổ phiếu OHLCV, thông tin doanh nghiệp niêm yết, chỉ số thị trường, dữ liệu realtime/order book và dữ liệu tin tức thị trường.

Tuần 1: Phân tích yêu cầu, khởi tạo hệ thống và ingest dữ liệu Bronze

Mục tiêu tuần 1

Hoàn thiện phạm vi dự án, xác định đầy đủ 5 nguồn dữ liệu, khởi tạo môi trường hệ thống và bắt đầu thu thập dữ liệu thô vào tầng Bronze.

Ngày	Công việc	Kết quả đầu ra
Ngày 1	Phân tích yêu cầu, xác định bài toán, use case, dashboard, phạm vi demo và 5 luồng dữ liệu chính	Hoàn thành phần đặt vấn đề, mục tiêu, input/output và phạm vi hệ thống
Ngày 2	Khởi tạo project, tạo cấu trúc thư mục, cấu hình môi trường Python, Docker Compose	Project chạy được ở mức cơ bản

Ngày 3	Cấu hình các service chính: MinIO, ClickHouse, Airflow, Kafka, Superset/Grafana	Môi trường local có đủ service cần thiết
Ngày 4	Xây dựng pipeline ingest dữ liệu OHLCV từ vnstock	Dữ liệu giá cổ phiếu thô được lưu vào Bronze
Ngày 5	Ingest thông tin doanh nghiệp niêm yết: ticker, tên công ty, sàn, ngành, shares_outstanding, P/E, EPS, ROE, ROA	Dữ liệu thô phục vụ <code>dim_stock</code> và <code>dim_sector</code>
Ngày 6	Ingest dữ liệu chỉ số thị trường: VN-Index, HNX-Index, VN30	Dữ liệu thô phục vụ <code>fact_market_index</code>
Ngày 7	Ingest thử dữ liệu tin tức từ Finnhub News hoặc RSS/HTML crawl, lưu title, content, url, published_at, source	Dữ liệu tin tức thô được lưu vào Bronze

Kết quả cuối tuần 1

- Hoàn thành phân tích yêu cầu và phạm vi hệ thống.
- Có môi trường chạy local bằng Docker.
- Có dữ liệu Bronze cho OHLCV.
- Có dữ liệu Bronze cho thông tin doanh nghiệp.
- Có dữ liệu Bronze cho chỉ số thị trường.
- Có dữ liệu Bronze cho tin tức.
- Cấu trúc lưu trữ trong MinIO được tổ chức theo nguồn dữ liệu và ngày xử lý.

Tuần 2: Xây dựng tầng Silver và kiểm soát chất lượng dữ liệu

Mục tiêu tuần 2

Làm sạch dữ liệu từ Bronze sang Silver, chuẩn hóa kiểu dữ liệu, xử lý lỗi và tích hợp Great Expectations để kiểm soát chất lượng dữ liệu.

Ngày	Công việc	Kết quả đầu ra
Ngày 8	Xây dựng script Bronze → Silver cho dữ liệu OHLCV bằng Polars	Dữ liệu OHLCV sạch ở tầng Silver
Ngày 9	Kiểm tra lỗi OHLCV: giá âm, volume âm/null, high < low, trùng ticker + ngày	Bộ dữ liệu giá đáng tin cậy hơn
Ngày 10	Xử lý Silver cho dữ liệu doanh nghiệp: chuẩn hóa ticker, tên công ty, ngành, sàn, shares_outstanding	Dữ liệu sạch phục vụ dimension
Ngày 11	Xử lý Silver cho dữ liệu chỉ số thị trường: chuẩn hóa ngày, điểm số, khối lượng, giá trị giao dịch	Dữ liệu sạch phục vụ market index
Ngày 12	Xử lý Silver cho tin tức: loại bài trùng, chuẩn hóa thời gian, bỏ bài quá ngắn, làm sạch HTML tags	Dữ liệu tin tức sạch ở Silver
Ngày 13	Tích hợp Great Expectations cho các pipeline chính: OHLCV, doanh nghiệp, index, tin tức	Bộ rule kiểm tra chất lượng dữ liệu
Ngày 14	Cấu hình Airflow DAG chạy tự động các bước Bronze → Silver, kiểm thử toàn bộ pipeline	Pipeline Silver chạy ổn định

Kết quả cuối tuần 2

- Có dữ liệu Silver cho OHLCV.
- Có dữ liệu Silver cho thông tin doanh nghiệp.
- Có dữ liệu Silver cho chỉ số thị trường.
- Có dữ liệu Silver cho tin tức.
- Có Great Expectations kiểm tra chất lượng dữ liệu.
- Airflow chạy được pipeline Bronze → Silver.
- Có log lỗi rõ ràng khi pipeline fail.

Tuần 3: Xây dựng tầng Gold, Star Schema, dbt và xử lý realtime

Mục tiêu tuần 3

Xây dựng tầng Gold trong ClickHouse, tạo Star Schema, tính toán chỉ báo kỹ thuật, tổng hợp dữ liệu tin tức/sentiment và thiết kế luồng realtime VWAP.

Ngày	Công việc	Kết quả đầu ra
Ngày 15	Thiết kế và tạo các bảng Dimension: <code>dim_date</code> , <code>dim_stock</code> , <code>dim_sector</code> , <code>dim_index</code>	Có các bảng dimension trong ClickHouse
Ngày 16	Tạo bảng <code>fact_daily_price</code> , cấu hình MergeTree, Partition, Order By	Có bảng fact trung tâm cho dữ liệu OHLCV
Ngày 17	Xây dựng dbt models tính SMA20, EMA12, EMA26, MACD, RSI14, Bollinger Band, market_cap	Có chỉ báo kỹ thuật trong <code>fact_daily_price</code>
Ngày 18	Xây dựng <code>fact_market_index</code> : VN-Index, HNX-Index, VN30, advance/decline ratio	Có dữ liệu Gold cho tổng quan thị trường
Ngày 19	Xử lý entity linking cho tin tức: gán bài viết với ticker bằng từ điển tên công ty/ticker và regex	Tin tức được gán với mã cổ phiếu
Ngày 20	Tính sentiment ở mức demo và tổng hợp vào <code>fact_news_sentiment_daily</code> : news_count, source_count, avg_sentiment_score, top_headline	Có dữ liệu Gold cho News & Sentiment
Ngày 21	Xây dựng thử luồng realtime: Kafka topic, ClickHouse Kafka Engine, Materialized View, <code>fact_realtime_vwap</code> hoặc dữ liệu giả lập	Có demo luồng realtime/VWAP cơ bản

Kết quả cuối tuần 3

- Có Star Schema hoàn chỉnh trong ClickHouse.
- Có `fact_daily_price` với các chỉ báo kỹ thuật.
- Có `fact_market_index` cho dashboard tổng quan.

- Có `fact_news_sentiment_daily` cho tin tức và sentiment.
- Có `dim_stock`, `dim_sector`, `dim_date`, `dim_index`.
- Có bản demo hoặc mô phỏng cho luồng realtime VWAP.
- Có dbt models và dbt tests cơ bản.

Tuần 4: Xây dựng dashboard, Alert Engine, Data Pipeline Monitor và hoàn thiện báo cáo

Mục tiêu tuần 4

Hoàn thiện phần hiển thị dữ liệu, cảnh báo, giám sát pipeline, báo cáo và kịch bản demo bảo vệ.

Ngày	Công việc	Kết quả đầu ra
Ngày 22	Kết nối Superset với ClickHouse, tạo dataset cho các bảng Gold	Superset đọc được dữ liệu ClickHouse
Ngày 23	Xây dựng dashboard Market Overview: VN-Index, top tăng/giảm, thanh khoản, heatmap ngành	Dashboard tổng quan thị trường
Ngày 24	Xây dựng dashboard Stock Detail: candlestick, volume, RSI, MACD, Bollinger Band, market cap	Dashboard chi tiết mã cổ phiếu
Ngày 25	Xây dựng Technical Signal Scanner: RSI quá mua/quá bán, breakout, breakdown, volume đột biến	Dashboard quét tín hiệu kỹ thuật
Ngày 26	Xây dựng Data Pipeline Monitor: Airflow DAG status, số lỗi GX/dbt, Kafka consumer lag, trạng thái service	Dashboard/section giám sát pipeline
Ngày 27	Xây dựng Alert Engine demo: <code>user_alerts</code> , Alert Checker, cooldown chống spam, ghi log vào <code>fact_alert_event</code>	Demo cảnh báo và lịch sử alert

Ngày 28 Hoàn thiện báo cáo, chỉnh format, chuẩn bị slide, chạy thử kịch bản demo cuối

Báo cáo, slide và demo hoàn chỉnh

Kết quả cuối tuần 4

- Có dashboard Market Overview.
- Có dashboard Stock Detail.
- Có dashboard Technical Signal Scanner.
- Có Data Pipeline Monitor.
- Có demo Alert Engine ở mức proof-of-concept.
- Có báo cáo hoàn chỉnh.
- Có slide và kịch bản demo bảo vệ.

Tổng hợp 5 luồng dữ liệu trong timeline

Luồng dữ liệu	Nguồn	Tuần xử lý chính	Kết quả
Giá cổ phiếu OHLCV	vnstock	Tuần 1, 2, 3	<code>fact_daily_price</code>
Thông tin doanh nghiệp	API niêm yết / Finnhub	Tuần 1, 2, 3	<code>dim_stock</code> , <code>dim_sector</code>
Chỉ số thị trường	VN-Index, HNX-Index, VN30	Tuần 1, 2, 3	<code>fact_market_index</code>
Tin tức thị trường	Finnhub News / RSS / HTML crawl	Tuần 1, 2, 3	<code>fact_news_sentiment_daily</code>
Dữ liệu realtime/order book	DNSE WebSocket API	Tuần 3, 4	<code>fact_realtime_vwap</code> , Alert Engine

Tổng hợp tiến độ theo tuần

Tuần n	Trọng tâm	Sản phẩm chính
Tuần 1	Phân tích yêu cầu, setup môi trường, ingest Bronze	Project structure, Docker Compose, MinIO Bronze, dữ liệu thô từ 4 nguồn batch/tin tức
Tuần 2	Làm sạch dữ liệu và Data Quality	Silver Layer, Polars transform, Great Expectations, Airflow DAG
Tuần 3	Gold Layer, Star Schema, dbt, realtime demo	ClickHouse, fact/dim tables, chỉ báo kỹ thuật, sentiment, VWAP demo
Tuần 4	Dashboard, Alert, Monitoring, báo cáo	Superset dashboard, Data Pipeline Monitor, Alert Engine demo, báo cáo và slide

Phạm vi ưu tiên nếu thời gian hạn chế

Mức độ ưu tiên	Hạng mục	Lý do
Bắt buộc	Pipeline OHLCV batch	Là nguồn dữ liệu chính của toàn bộ hệ thống
Bắt buộc	Bronze/Silver/Gold	Thể hiện rõ kiến trúc Data Lakehouse
Bắt buộc	Great Expectations	Chứng minh hệ thống có kiểm soát chất lượng dữ liệu
Bắt buộc	ClickHouse + Star Schema	Là tầng phục vụ truy vấn và dashboard
Bắt buộc	Chỉ báo kỹ thuật	Là giá trị phân tích chính của hệ thống
Bắt buộc	Market Overview	Cho thấy toàn cảnh thị trường
Bắt buộc	Stock Detail	Cho thấy năng lực phân tích từng mã
Bắt buộc	Technical Signal Scanner	Tạo insight từ dữ liệu thay vì chỉ hiển thị dữ liệu

Bắt buộc	Data Pipeline Monitor	Thể hiện điểm khác biệt của Data Lakehouse và DataOps
Nên có	Tin tức & Sentiment	Thể hiện khả năng tích hợp dữ liệu phi cấu trúc
Nên có	Realtime VWAP	Thể hiện streaming analytics nhưng phụ thuộc nguồn realtime
Nâng cao	Alert Engine	Hoàn thiện use case cảnh báo cá nhân hóa
Nâng cao	Text-to-SQL	Là proof-of-concept nếu còn thời gian

Kịch bản demo cuối cùng

Khi bảo vệ, hệ thống có thể demo theo trình tự sau:

1. Mở Airflow để cho thấy pipeline ingest dữ liệu chạy thành công.
2. Mở MinIO để minh họa dữ liệu được lưu ở Bronze và Silver.
3. Mở Great Expectations hoặc log validation để chứng minh dữ liệu được kiểm tra chất lượng.
4. Mở ClickHouse để cho thấy dữ liệu Gold đã được tính toán thành các bảng fact/dimension.
5. Mở Superset dashboard Market Overview để xem toàn cảnh thị trường.
6. Chọn một mã cổ phiếu và drill-down sang Stock Detail.
7. Mở Technical Signal Scanner để xem các mã có tín hiệu RSI, MACD, Bollinger Band.
8. Mở News & Sentiment để minh họa dữ liệu tin tức đã được tổng hợp.
9. Mở Data Pipeline Monitor để chứng minh hệ thống có giám sát pipeline.
10. Nếu kịp, demo thêm Realtime VWAP hoặc Alert Engine.